

Week 07

Transaction and Isolation

Cảm ơn thầy Trần Duy Quang đã cung cấp template cho môn học
Cảm ơn thầy Hồ Tuấn Thanh và thầy Đỗ Nguyên Kha đã xây dựng nội dung thực hành



Lecturer: **TS. Lê Thị Nhân.**
Lab TA: **ThS. Nguyễn Hải Đăng.**

1

Notes

Put **ALL** SQL statements to **ONE** text (SQL) document names student ID 2012xxx.sql and a word/pdf document answer the questions 2012xxx.doc/docx/pdf then put into a zip/rar 2012xxx_yy.zip/rar (yy is the number of requirements you finished):

- Comment Student ID and Full name on top of the file
- Run ALL lines in file without any error
- USE the CORRECT database

2

Sample Bank database

```
CREATE DATABASE Bank;

GO

USE Bank;

CREATE TABLE Account (
    Id INT PRIMARY KEY,
    [Name] NVARCHAR(50),
    Balance INT
);

INSERT INTO Account(Id, [Name], Balance) VALUES (1, N'VTL', 100000);
INSERT INTO Account(Id, [Name], Balance) VALUES (2, N'TTV', 100000000);
```

Commit/Rollback

Example: Transfer 100.000 unit from TTV to VTL

```
BEGIN TRANSACTION
UPDATE Account SET Balance = Balance - 100000 WHERE Id = 2;
UPDATE Account SET Balance = Balance + 100000 WHERE Id = 1;
COMMIT;
```

COMMIT applies changes into database

```
BEGIN TRANSACTION
UPDATE Account SET Balance = Balance - 100000 WHERE Id = 2;
UPDATE Account SET Balance = Balance + 100000 WHERE Id = 1;
ROLLBACK;
```

ROLLBACK discards the changes from database

Try/Catch

The commit/rollback usually are usually with try/catch

```
BEGIN TRANSACTION
```

```
BEGIN TRY
    UPDATE Account SET Balance = Balance - 100000 WHERE Id = 2;
    UPDATE Account SET Balance = Balance + 100000 WHERE Id = 1;
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    THROW 50000, 'Cannot transfer', 1
END CATCH;
```

Isolation level

There are 4 standard isolation level: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ and SERIALIZABLE. SQL Server also has the 5th isolation level SNAPSHOT which is not mentioned in this lab.

Let's try each isolation level with the transfer procedure. Note: The default isolation level is READ COMMITTED, thus dirty read cannot happen.

READ UNCOMMITTED

Create the transfer stored procedure

```
CREATE PROCEDURE sp_transfer(@fromAccountId int, @toAccountId int, @amount int)
AS
BEGIN TRANSACTION
BEGIN TRY
    DECLARE @fee INT;
    SET @fee = 10;
    -- Check if from account exists
    IF NOT EXISTS(SELECT Id FROM Account WHERE Id = @fromAccountId)
    BEGIN
        THROW 50001, 'From account does not exist', 1;
    END
    -- Check if from account has enough balance
    IF NOT EXISTS(SELECT Id FROM Account WHERE Id = @fromAccountId AND Balance >= @amount + @fee)
    BEGIN
        THROW 50002, 'From account does not have enough balance', 1;
    END
END TRY
END CATCH;
```

```
END
-- Check if to account exist
IF NOT EXISTS(SELECT Id FROM Account WHERE Id = @toAccountId)
BEGIN
    THROW 50003, 'To account does not exist', 1;
END
-- Check if same account
IF @toAccountId = @fromAccountId
BEGIN
    THROW 50004, 'Cannot transfer to same account', 1;
END
-- Debit from account
UPDATE Account SET Balance = Balance - @amount WHERE Id = @fromAccountId;

-- Wait for 5 second
WAITFOR DELAY '00:00:05';

-- Debit fee from account
UPDATE Account SET Balance = Balance - @fee WHERE Id = @fromAccountId;
-- Credit to account
UPDATE Account SET Balance = Balance + @amount WHERE Id = @toAccountId;
PRINT 'Transfer completed ' + CAST(@amount AS NVARCHAR(20)) + ' from account ' + CAST(@fromAccountId AS
NVARCHAR(20)) + ' to account ' + CAST(@toAccountId AS NVARCHAR(20));
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    THROW 50005, 'Transaction error!', 1;
END CATCH;
GO
```

The WAITFOR DELAY simulates the real scenario when the database may be busy and execute the procedure too long thus making the concurrent problem easy to produce.

Try to execute the sp_transfer first with valid parameters. It would take at least 5 seconds to complete:

```
EXECUTE sp_transfer 2, 1, 10000;
```

While the above procedure is running, open the new Window/Connection to database and execute the following statements:

```
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
SELECT Balance FROM Account WHERE Id = 2;
COMMIT TRANSACTION;
```

Remember the account 2's balance and try to run again after the transfer procedure is completed. You would see the difference between 2 times getting the balance of account because the transaction has READ UNCOMMITTED isolation level, it's read the dirty value of balance because the procedure didn't commit.

READ COMMITED

```
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL READ COMMITTED
SELECT Balance FROM Account WHERE Id = 2;
COMMIT TRANSACTION;
```

Try again with the READ COMMITTED (or higher level). The transaction should wait for the procedure completed then return the latest balance account.

READ COMMITED ensured all the data is committed before read but not ensure it should read the same value as previous statement in the transaction because other transactions may commit new value.

Remove the WAITFOR DELAY in the sp_transfer method for faster running. Try the follow scenario:

1. Execute the sp_transfer
2. Run the following transaction:

```
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL READ COMMITTED
SELECT Balance FROM Account WHERE Id = 2;
-- Wait for 5 second
WAITFOR DELAY '00:00:05';
SELECT Balance FROM Account WHERE Id = 2;
COMMIT TRANSACTION;
```

3. While the above transaction is running, try to execute the sp_transfer again

You would see the values of the same select statement is different because the sp_transfer commit new balance before the second select statement is executed.

REPEATABLE READ

```
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ
SELECT Balance FROM Account WHERE Id = 2;
-- Wait for 5 second
WAITFOR DELAY '00:00:05';
SELECT Balance FROM Account WHERE Id = 2;
COMMIT TRANSACTION;
```

Change the isolation level to REPEATABLE READ and try the previous scenario again. The second SELECT should output the same balance as the first SELECT.

REPEATABLE READ ensures the statement with the same criteria output the same value because other process is blocked from modifying (or removing) the data which matched the criteria. They should wait for the REPEATABLE READ transaction to complete to update data.

The repeatable read doesn't guarantee the new data that also match the criteria not appear in the second call (insert new row). Let's create a new procedure sp_createAccount which takes the account name and automatically generate the new id for that account by increasing the latest account by 1:

```
CREATE PROCEDURE sp_createAccount(@name nvarchar(50))
AS
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ
BEGIN TRY
    DECLARE @latestId int;
    -- Check the account table (demo)
    SELECT * FROM Account;

    IF NOT EXISTS(SELECT TOP 1 Id FROM Account)
        SET @latestId = 0;
    ELSE
        SELECT TOP 1 @latestId = Id FROM Account ORDER BY Id DESC;
```

```
SET @latestId = @latestId + 1;
-- Wait for 5 second
WAITFOR DELAY '00:00:05';

-- Check the account table (demo)
SELECT * FROM Account;

INSERT INTO Account(Id, [Name], Balance) VALUES(@latestId, @name, 0);

-- Check the account table (demo)
SELECT * FROM Account;

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    THROW;
END CATCH;
GO
```

Try the following scenario:

1. Create new account with name “NPT”
2. While the first procedure is not complete, try to create new account with name “NXP”

The first create account should complete without error and the second would fail because it didn't aware of new row had been inserted. In the log of creating account for “NXP”, you can see the second select statement show 1 more row than the first.

SERIALIZABLE

This isolation level ensures all parallel transactions are executed as they run one by one (in an order). Let's change the above example to this isolation level. The latter would fail due to the row with the same id was locked by the first one. When two processes try to lock the same resource, 1 of them would be chosen to die.

3 Assignments

1. Lab

YY: 04 (4 isolation level)

Database name: Bank

- Try the tutorial and write the result (screenshot) of each case in word document

2. Homework

YY: 03 (3 stored procedures + a PDF file explains how you apply isolation levels)

Database name: Shop

- **Product:**
 - Id: int, primary key
 - Name: nvarchar(50), not null
 - Description: text
 - Price: int, not null
 - Remain: int, not null, default 0
 - Sold: int, not null, default 0
- **Order:**
 - Id: int, primary key
 - Total: int, not null, default 0
 - Customer: nvarchar(50)
- **OrderDetail:**
 - OrderId: int, primary key, not null, foreign key
 - ProductId: int, primary key, not null, foreign key
 - Price: int, not null
 - Quantity: int, not null, > 0

Create SQL file with requirements:

1. Create database
2. Create tables
3. Create stored procedures:
 - a. sp_createEmptyOrder: Create new order for a customer (name) with total = 0
 - b. sp_addItemToOrder: Add a product (by productId) with Quantity as a parameter
 - i. Check product exists
 - ii. Check order exists
 - iii. Check order already has this product (decide to insert or update order detail)
 - iv. Check Remain of product $\geq$ Quantity
 - v. Update Remain = Remain – Quantity, Sold = Sold + Quantity

- vi. Add order detail
- c. sp_removeItemFromOrder: Remove a product (by productId) in an order (by orderId)
 - i. Check product exists
 - ii. Check order exists
 - iii. Check order detail exists
 - iv. Update Remain = Remain + Quantity, Sold = Sold - Quantity
 - v. Remove order detail

Use transaction to ensure the stored procedures work as requirements (the consistent of Sold and Remain). Place WAITFOR DELAY and screenshot the result of concurrency simulation in word document.

Notes:

- What would happen if 2 people add the last remaining product at the same time to their order?
- What would happen if 1 person add and 1 person remove product to/from their order?
- ...